

A Self-Healing, Agentic Retrieval-Augmented Generation System

Architecture, Related Work, and Design Rationale

Parth

June 2026

Abstract

Large Language Models (LLMs) are highly capable text generators but remain prone to hallucination—producing fluent, confident output that is factually incorrect or unsupported by evidence. Retrieval-Augmented Generation (RAG) mitigates this by grounding generation in externally retrieved documents, but standard RAG pipelines assume retrieval is always accurate, which is frequently not the case in practice. This paper documents the design of a self-healing RAG chatbot that addresses this assumption through retrieval grading, automatic query refinement, neural reranking, and a corrective web-search fallback, all orchestrated as a LangGraph agentic system. We review the core research that informs each architectural decision, explain why each technique was selected over alternatives, and outline the evaluation strategy used to validate the system.

1. Introduction

Most RAG tutorials implement a “naive” pipeline: embed a query, retrieve the top-k chunks, stuff them into a prompt, and generate an answer. This works well when retrieval is good and fails silently when it isn’t — the LLM will still generate a confident answer even from irrelevant or low-quality context. This system was built specifically to address this failure mode, moving from a linear LCEL chain toward an agentic graph that can detect poor retrieval, retry it, refine the query, or fall back to live web search when the internal knowledge base cannot answer the question.

This document is structured as a literature-grounded justification for each component of that system, rather than a pure implementation log. Section 2 reviews the foundational and supporting research. Section 3 maps each piece of research onto a specific node in the system architecture. Section 4 explains the implementation stack. Section 5 outlines evaluation. Section 6 discusses limitations.

2. Related Work

2.1 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)

Lewis et al. (2020), in their NeurIPS paper “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” introduced the original RAG formulation: combining a pre-trained parametric language model with a non-parametric, retrievable memory (a dense vector index) to ground generation in external knowledge. This established the basic two-stage pattern — retrieve, then generate — that nearly all modern RAG systems build upon.

2.2 Corrective Retrieval Augmented Generation (Yan et al., 2024)

Yan et al. (2024) identified the central weakness of standard RAG: it has no mechanism to detect or correct bad retrieval. Their proposed method, Corrective RAG (CRAG), introduces a lightweight evaluator that scores the quality of retrieved documents and triggers one of three actions depending on a confidence threshold: accept the documents, discard and refine them, or discard them entirely and fall back to web search. This is the single most directly applicable paper to this system’s design — the retrieval-grade-then-retry-or-search-web pattern in CRAG is structurally identical to this system’s intended corrective loop, with Tavily standing in for the web search component.

2.3 Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection (Asai et al., 2023)

Asai et al. (2023) extended the idea of self-correction beyond retrieval into generation itself. Their Self-RAG framework trains a model to emit “reflection tokens” that judge whether retrieval was necessary, whether the retrieved passages actually support the generated claims, and whether the final output is useful. While this system does not train a custom model with reflection tokens, this paper provides the conceptual justification for adding a dedicated hallucination-grading node after generation — checking not just “was retrieval good?” but “is the answer actually supported by what was retrieved?”

2.4 Passage Re-ranking with BERT (Nogueira & Cho, 2019)

Nogueira and Cho (2019) demonstrated that a BERT-based cross-encoder, applied as a second-stage scorer after an initial fast retriever (originally BM25), produces substantially better passage rankings than the first-stage retriever alone — because it can model fine-grained interactions between the full query and each candidate passage rather than relying on vector similarity alone. This is the architectural ancestor

of the reranker node added to this system: Pinecone’s cosine similarity search acts as the cheap first-stage retriever, and a dedicated reranker (e.g. Cohere Rerank or bge-reranker) re-scores the top candidates before they reach the grading and generation stages.

2.5 RAGAS: Automated Evaluation of Retrieval Augmented Generation (Es et al., 2023/2024)

Es et al. (2023/2024) addressed a gap that most RAG projects ignore entirely: how do you know if your RAG system is actually good, without hand-labeling a test set? Their framework, RAGAS, provides reference-free metrics — faithfulness, answer relevancy, context precision, and context recall — that can be computed automatically using an LLM as a judge. This gives the system a way to produce a quantified, defensible claim (e.g. “94% faithfulness on a held-out test set”) rather than a purely qualitative one.

2.6 Large Language Models Hallucination: A Comprehensive Survey (Alansari & Luqman, 2025)

Alansari and Luqman (2025) provide a broad survey of why LLMs hallucinate and how hallucination can be detected, spanning the full model lifecycle from training data to inference-time behavior. This survey is used here not as a direct architectural blueprint but as supporting evidence for why a dedicated hallucination-grading step is a recognized, research-backed mitigation strategy rather than an ad hoc addition.

3. System Architecture and Mapping to Research

The system’s LangGraph implementation is organized as a directed graph of nodes connected by conditional edges. The table below maps each node to the research that motivates it.

Node	Function	Research Basis
Query Router	Decides whether a query needs retrieval at all	Self-RAG (adaptive retrieval decision)
Retriever	Pulls top-k chunks from Pinecone	Lewis et al. 2020 (base RAG retrieval)
Reranker	Re-scores retrieved chunks for relevance	Nogueira & Cho 2019 (cross-encoder reranking)
Retrieval Grader	Scores chunk relevance/quality, decides accept/retry/web-search	CRAG (Yan et al. 2024)

Node	Function	Research Basis
Query Refiner	Rewrites the query if retrieval was graded poor	CRAG (corrective query refinement)
Web Search Fallback	Tavily search when internal retrieval is insufficient	CRAG (web search as corrective action)
Generator	Produces the answer from graded/reranked context	Lewis et al. 2020
Hallucination Grader	Checks if the generated answer is supported by retrieved context	Self-RAG (self-critique of generation)
Observability Layer	Traces every node hop, latency, and token cost	N/A — engineering layer (LangSmith)

This structure means every conditional branch in the graph corresponds to a decision point that has been independently studied and validated in the literature, rather than being an arbitrary design choice.

4. Implementation Stack

- **Orchestration:** LangGraph (StateGraph, conditional edges)
- **LLM & Embeddings:** Mistral AI (mistral-large-latest, mistral-embed)
- **Vector Store:** Pinecone (index: rag-chatbot, 1024-dim, cosine similarity, AWS us-east-1)
- **Reranker:** Cohere Rerank or bge-reranker (cross-encoder, local/HuggingFace)
- **Web Search Fallback:** Tavily API
- **Observability:** LangSmith
- **Evaluation:** RAGAS
- **Serving:** FastAPI with Server-Sent Events (SSE) for streaming responses

5. Evaluation Strategy

The system will be evaluated using RAGAS on a held-out set of question-context-answer triples constructed from the same document sources used for ingestion (PDF, URL, YouTube transcript, plain text). The primary metrics tracked are faithfulness (does the answer follow from the retrieved context), answer relevancy (does the answer address the question), context precision (how much of the retrieved context is actually relevant), and context recall (how much of the necessary information was

retrieved at all). These metrics will be reported before and after the addition of the reranker and corrective loop to demonstrate measurable improvement, rather than relying on qualitative impressions alone.

6. Limitations and Future Work

The current design qualifies as addressing retrieval-level correction (grading, refinement, reranking, web fallback) and generation-level self-checking (hallucination grading), but does not yet implement a closed-loop answer-rewriting step where a failed hallucination check triggers full regeneration with a different strategy — this is a natural extension suggested by the Self-RAG line of work. Additionally, the retrieval evaluator and hallucination grader currently rely on LLM-as-judge scoring rather than a dedicated lightweight classifier as in the original CRAG paper; a smaller, fine-tuned evaluator could reduce latency and cost at scale.

7. Conclusion

This system’s evolution from a linear LCEL chain into a LangGraph agentic system is grounded in a specific, well-documented line of RAG research: foundational retrieval-augmented generation, corrective retrieval loops, self-reflective generation, neural reranking, and reference-free evaluation. Framing the project this way — as an implementation of published, peer-reviewed ideas rather than an ad hoc tutorial project — is what distinguishes it as a credible portfolio piece for AI engineering roles.

References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
2. Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. *arXiv preprint*. <https://arxiv.org/abs/2401.15884>
3. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint*. <https://arxiv.org/abs/2310.11511>

4. Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint*. <https://arxiv.org/abs/1901.04085>
5. Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 150–158. <https://arxiv.org/abs/2309.15217>
6. Alansari, A., & Luqman, H. (2025). Large Language Models Hallucination: A Comprehensive Survey. *arXiv preprint*. <https://arxiv.org/abs/2510.06265>